

Algorithm	Best	Average	Worst	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	✓
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	✗
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	✓
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	✓
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	✗
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	✗

S.No	Module Topic	LeetCode No.	Problem Name	Difficulty	Main Concept
1	Bubble Sort	88	Merge Sorted Array	Easy	Array Merging, Basic Sorting
2	Bubble Sort	75	Sort Colors	Medium	In-place Sorting (can be solved using Bubble Sort for beginners)
3	Selection Sort	977	Squares of a Sorted Array	Easy	Sorting & Two Pointers
4	Selection Sort	561	Array Partition	Easy	Sorting Before Greedy Selection
5	Insertion Sort	147	Insertion Sort List	Medium	Insertion Sort on Linked List
6	Insertion Sort	912	Sort an Array	Medium	Implement Sorting Algorithms (Insertion Sort for small inputs)
7	Merge Sort	912	Sort an Array	Medium	Merge Sort Implementation
8	Merge Sort	148	Sort List	Medium	Merge Sort on Linked List
9	Merge Sort	315	Count of Smaller Numbers After Self	Hard	Modified Merge Sort

10	Merge Sort	493	Reverse Pairs	Hard	Divide & Conquer, Merge Sort
11	Quick Sort	912	Sort an Array	Medium	Quick Sort Implementation
12	Quick Sort	215	Kth Largest Element in an Array	Medium	Quick Select (Quick Sort Partition)
13	Quick Sort	75	Sort Colors	Medium	Partitioning Technique
14	Heap Sort	215	Kth Largest Element in an Array	Medium	Heap / Priority Queue
15	Heap Sort	703	Kth Largest Element in a Stream	Easy	Min Heap
16	Heap Sort	1046	Last Stone Weight	Easy	Max Heap
17	Heap Sort	347	Top K Frequent Elements	Medium	Heap & Frequency
18	Heap Sort	973	K Closest Points to Origin	Medium	Heap / Priority Queue
19	Comparative Analysis	912	Sort an Array	Medium	Compare Merge, Quick, Heap, Insertion Sort
20	Comparative Analysis	164	Maximum Gap	Medium	Sorting Algorithm Analysis

Step 1:Read the first sorted array `nums1`.

Step 2:Read the second sorted array `nums2`.

Step 3:Create a new array `result`.

Step 4:Initialize three variables:

- `i = 0` → First array index

- $j = 0 \rightarrow$ Second array index
- $k = 0 \rightarrow$ Result array index

Step 5: Compare elements of both arrays.

- If $nums1[i] < nums2[j]$
 - Copy $nums1[i]$ to $result[k]$
 - Increment i
- Else
 - Copy $nums2[j]$
 - Increment j

Increment k .

Step 6: Copy remaining elements from $nums1$.

Step 7: Copy remaining elements from $nums2$.

Step 8: Copy all elements from $result$ back into $nums1$.

Step 9: Print the merged array.

Squares of a Sorted Array

Given an integer array $nums$ sorted in **non-decreasing** order, return *an array of the squares of each number sorted in non-decreasing order*.

Example 1:

Input: $nums = [-4, -1, 0, 3, 10]$

Output: $[0, 1, 9, 16, 100]$

Explanation: After squaring, the array becomes $[16, 1, 0, 9, 100]$.
After sorting, it becomes $[0, 1, 9, 16, 100]$.

Example 2:

Input: $nums = [-7, -3, 2, 3, 11]$

Output: $[4, 9, 9, 49, 121]$

Step 1: Read the sorted array.

Step 2: Find the square of every element.

Step 3: Store the squared values in a new array.

Step 4: Sort the squared array using **Bubble Sort**.

Step 5: Print the sorted squared array.

```
#include <stdio.h>

int main()
{
    int nums[] = {-4, -1, 0, 3, 10};
    int n = 5;

    int result[5]; // for storing the squares of the elements

    int i, j, temp;

    // Step 1: Square every element
    for(i = 0; i < n; i++)
    {
        result[i] = nums[i] * nums[i];
    }

    // Step 2: Bubble Sort
    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++) // here n-i-1 is used to avoid
unnecessary comparisons of already sorted elements
        {
            if(result[j] > result[j + 1])
            {
                temp = result[j];
                result[j] = result[j + 1];
                result[j + 1] = temp;
            }
        }
    }
}
```

```

    }

}

// Step 3: Print the sorted squares
printf("Sorted Squares:\n");

for(i = 0; i < n; i++)
{
    printf("%d ", result[i]);
}

return 0;
}

```

912. Sort an Array

Given an array of integers `nums`, sort the array in ascending order and return it.

You must solve the problem **without using any built-in** functions in $O(n\log(n))$ time complexity and with the smallest space complexity possible.

Example 1:

Input: `nums = [5,2,3,1]`

Output: `[1,2,3,5]`

Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).

Example 2:

Input: `nums = [5,1,1,2,0,0]`

Output: `[0,0,1,1,2,5]`

Explanation: Note that the values of `nums` are not necessarily unique.

Steps :

Step 1 Read the array.

Step 2 Divide the array into two equal halves.

Step 3 Keep dividing until each sub-array contains only one element.

Step 4 Merge two sorted sub-arrays into one sorted array.

Step 5 Repeat merging until the complete array becomes sorted.

Step 6 Print the sorted array.

315. Count of Smaller Numbers After Self

Given an integer array *nums*, return *an integer array counts* where *counts[i]* is the number of smaller elements to the right of *nums[i]*.

Example 1:

Input: *nums* = [5,2,6,1]

Output: [2,1,1,0]

Explanation:

To the right of 5 there are **2** smaller elements (2 and 1).

To the right of 2 there is only **1** smaller element (1).

To the right of 6 there is **1** smaller element (1).

To the right of 1 there is **0** smaller element.

Example 2:

Input: *nums* = [-1]

Output: [0]

Example 3:

Input: *nums* = [-1,-1]

Output: [0,0]

Step 1 Read the input array.

Step 2 Create another array `count[ ]`.

Step 3 For every element in the array:

- Initialize `count = 0`.

Step 4 Check every element on its right.

Step 5 If the right element is smaller,

- Increase the count.

Step 6 Store the count in the result array.

Step 7 Print the result array.

```
// Step 1: Divide the array into two halves recursively.
// Step 2: Continue dividing until each sub-array contains only one element.
// Step 3: Merge the sorted sub-arrays.
// Step 4: During the merge process, whenever an element from the right
sub-array
//is placed before an element in the left sub-array, it indicates a smaller
element to the right.
// Step 5: Count the number of such smaller elements and update the
corresponding
//index of the original array.
// Step 6: Continue merging until the complete array is sorted.
// Step 7: Return the count array.
```

```
#include <stdio.h>
```

```
#define MAX 100
```

```
// Structure to store value and its original index
typedef struct
{
    int value;
```

```

    int index;
} Node; // node is a structure that holds the value of an element and its
original index in the array

Node arr[MAX];
Node temp[MAX];

int count[MAX];

// Merge two sorted halves and update counts
void merge(int left, int mid, int right) // a recursive function that merges
two sorted halves of the array and updates the count of smaller elements to
the right
{
    int i = left;
    int j = mid + 1;
    int k = left;

    int rightCount = 0;

    while(i <= mid && j <= right)
        // here we are comparing the elements of the left and right halves of the
        array to merge them in sorted order
        {
            if(arr[j].value < arr[i].value)
            {
                temp[k++] = arr[j++];
                rightCount++; // for every element in the right half that is
smaller than the current element in the left half, we increment the
rightCount
            }
            else
            {
                count[arr[i].index] += rightCount;
                temp[k++] = arr[i++];
            }
        }

    // Copy remaining elements of left half

```



```

while(i <= mid)
{
    count[arr[i].index] += rightCount;
    temp[k++] = arr[i++];
}

// Copy remaining elements of right half
while(j <= right)
{
    temp[k++] = arr[j++];
}

// Copy back to original array
for(i = left; i <= right; i++)
{
    arr[i] = temp[i];
}
}

// Recursive Merge Sort
void mergeSort(int left, int right) // calling it aGain to sort the left and
right halves of the array recursively and merge them while counting the
smaller elements to the right
{
    if(left >= right)
        return;

    int mid = (left + right) / 2;

    mergeSort(left, mid); // CALLING THE FUNCTION RECURSIVELY TO SORT THE
LEFT HALF OF THE ARRAY
    mergeSort(mid + 1, right); // CALLING THE FUNCTION RECURSIVELY TO SORT
THE RIGHT HALF OF THE ARRAY

    merge(left, mid, right); // MERGING THE TWO SORTED HALVES AND UPDATING
THE COUNT OF SMALLER ELEMENTS TO THE RIGHT
}

int main()

```

```

{
    int nums[] = {5, 2, 6, 1};
    int n = 4;

    int i;

    // Initialize array with values and original indices
    for(i = 0; i < n; i++)
    {
        arr[i].value = nums[i];
        arr[i].index = i;
        count[i] = 0;
    }

    // Perform Merge Sort
    mergeSort(0, n - 1);

    // Print result
    printf("Count Array:\n");

    for(i = 0; i < n; i++)
    {
        printf("%d ", count[i]);
    }

    return 0;
}

```

215. Kth Largest Element in an Array

Given an integer array `nums` and an integer `k`, return *the* k^{th} *largest element in the array*.

Note that it is the k^{th} largest element in the sorted order, not the k^{th} distinct element.

Can you solve it without sorting?

Example 1:

Input: `nums = [3,2,1,5,6,4]`, `k = 2`

Output: 5

Example 2:

Input: nums = [3,2,3,1,2,4,5,5,6], k = 4

Output: 4

Steps

Using Quick Sort

Algorithm

Step 1 Read the array.

Step 2 Choose a pivot element.

Step 3 Partition the array so that:

- Smaller elements are placed on the left.
- Larger elements are placed on the right.

Step 4 Recursively sort the left and right sub-arrays.

Step 5 After sorting, the kth largest element is located at:

Index = $n - k$

Step 6 Print `nums[n-k]`.

Example

Input

nums = [3,2,1,5,6,4]

k = 2

After Quick Sort

1 2 3 4 5 6

Index = $6 - 2 = 4$

Answer

5

```
// Swap two elements
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Partition function
int partition(int nums[], int low, int high)
{
    int pivot = nums[high];
    int i = low - 1;

    for(int j = low; j < high; j++)
    {
        if(nums[j] < pivot)
        {
            i++;
            swap(&nums[i], &nums[j]);
        }
    }

    swap(&nums[i + 1], &nums[high]);

    return i + 1;
}

// Quick Sort
void quickSort(int nums[], int low, int high)
{
    if(low < high)
    {
        int pi = partition(nums, low, high);

        quickSort(nums, low, pi - 1);
    }
}
```

```

        quickSort(nums, pi + 1, high);
    }
}

// LeetCode Function do no change
int findKthLargest(int* nums, int numsSize, int k)
{
    quickSort(nums, 0, numsSize - 1);

    return nums[numsSize - k];
}

```

```

#include <stdio.h>

void swap(int *a, int *b) // a recursive function that swaps the values of
two integers using pointers
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) // recursive function that
partitions the array into two halves based on a pivot element and returns the
index of the pivot after partitioning
//here high n low are the indices of the sub-array that we want to partition,
and arr is the array that we want to sort
{
    int pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++)
    {
        if(arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
}

```

```

        swap(&arr[i + 1], &arr[high]);

        return i + 1;
    }

void quickSort(int arr[], int low, int high) // here we are implementing the
quick sort algorithm recursively to sort the array in ascending order
{
    if(low < high)
    {
        int pi = partition(arr, low, high); // calling the partition function
to get the index of the pivot element after partitioning the array

        quickSort(arr, low, pi - 1); // calling the quickSort function
recursively to sort the left half of the array
        quickSort(arr, pi + 1, high); // calling the quickSort function
recursively to sort the right half of the array
    }
}

int main()
{
    int nums[] = {3,2,1,5,6,4};

    int n = 6;

    int k = 2;

    quickSort(nums, 0, n - 1);

    printf("Sorted Array:\n");

    for(int i = 0; i < n; i++)
        printf("%d ", nums[i]);

    printf("\n\n%dth Largest Element = %d", k, nums[n-k]); // here we are
printing the kth largest element in the sorted array by accessing the element
at index n-k, where n is the size of the array and k is the position of the

```

```
largest element we want to find
```

```
    return 0;  
}
```

Heap Sort process
4 10 3 5 1

Step 1: Build Max Heap	<pre> 10 /\n 5 3 /\n 4 /\n1</pre>
Array Representation	10 5 3 4 1
Swap Root and Last Element	10 5 3 4 1 ↓ 1 5 3 4 10
Heapify	5 4 3 1 10
Swap Again	5 4 3 1 10 ↓ 1 4 3 5 10
Heapify	4 1 3 5 10
	1 3 4 5 10

Operation	Complexity
Build Max Heap	$O(n)$
Heapify (Each Extraction)	$O(\log n)$
Total Heap Sort	$O(n \log n)$